

Design Document

Music Works Catalogue Portal

Computer Science Final Year Project

1. Project Overview

This project is based on the "Working with Works — A New Approach to Music Cataloguing" template. It delivers a polished, professional web portal for exploring thematic catalogues of classical composers' works currently encoded in MEI XML. The original publication system — MerMEId, built by the Danish Royal Library in 2014 — has broken down, leaving valuable scholarly data inaccessible. This project transforms that data into a queryable format, exposes it through a documented REST API built to professional standards, and presents it through an interactive, accessible web interface. A novel Works Discovery Engine, driven directly by user needs identified in the research, surfaces musical connections between works that no existing tool reveals.

2. Domain and Users

The domain is digital musicology — the scholarly cataloguing of classical music works. A thematic catalogue is a systematic scholarly record of every work by a composer, including manuscripts, sources, performance history, instrumentation, and musical incipits (the opening notes of each work). This is a niche but well-established academic field with real, identifiable users and a documented, unmet need.

- Musicologists and scholars — need structured, searchable access to catalogue data for research.
- Music librarians — maintain and publish catalogues; need sustainable, open infrastructure.
- Music students and enthusiasts — want to explore a composer's full output without specialist XML knowledge.

- Developers and digital humanities researchers — need a documented public API to build new tools.

These groups span a wide range of technical ability — the central design tension. The system must serve a non-technical student and a developer consuming structured JSON equally well, which drives the architectural separation between the API and the frontend.

3. Problems Solved, and the Research Behind Them

Each problem this project addresses is grounded in published research and the documented history of the MerMEId system. The table below maps user problems to their evidence base and to the specific design response.

User problem	Evidence / research	How the design solves it
Catalogue data is inaccessible — the publication interface has broken	Bue et al. (2023): MerMEId's publication pipeline ceased functioning after institutional transition	Modern, independently-deployable web portal + REST API rebuilt on a sustainable stack
Data is only usable by XML specialists	Krabbe & Geertinger (2012): catalogues are full of 'esoteric abbreviations' usable only by specialists	Intuitive web UI with search, filtering and browsing — no XML knowledge required
No machine-readable access for tool-building	Weigl et al. (2019): scholarship needs interoperable, machine-readable, FAIR music data	Documented JSON REST API with linked-data identifiers (Wikidata, MusicBrainz)
Systems fail and disappear over time	Mathiak et al. (2020): 376 of 466 digital editions vanished; avg. lifetime 8.5 years	Simple mainstream stack, open standards, Docker, automated tests — no institutional lock-in
Catalogues are siloed; no cross-composer exploration	Krabbe & Geertinger (2012): interoperability between catalogues is an unsolved challenge	Catalogue-agnostic pipeline + Works Discovery Engine surfacing cross-composer connections

The novel feature of the project — the Works Discovery Engine — is not arbitrary: it responds directly to the interoperability gap identified by Krabbe & Geertinger (2012) and the call for richer, multi-faceted exploration in Weigl et al. (2019). By computing melodic similarity from MEI incipits, it surfaces connections across composers that the source catalogues encode but never expose to users.

4. Justification of Design Choices

- Separating the API from the frontend serves the two distinct audiences and is the single most consequential decision — the API is a first-class product, not an afterthought.
- Transforming MEI XML into SQLite addresses the access barrier directly: SQL is queryable by standard tools, whereas the original eXist-db/XQuery stack required specialist knowledge and recreated MerMEId's fragility.
- Preserving the original MEI XML as the source of truth makes the transformation reversible — no scholarly data is lost and future migration to RDF remains open.
- A catalogue-agnostic pipeline reflects the domain reality that many composer catalogues share the MEI format, and supports exploring multiple catalogues together.
- A mainstream, simple, well-tested stack over the ideal Linked Data approach is justified by sustainability research (Mathiak et al., 2020): single-institution dependency and obsolete stacks are the primary causes of digital-humanities software failure.

5. System Architecture

The system follows a professional layered pipeline architecture with strict separation of concerns. Data flows in one direction from source MEI XML, through a transformation pipeline, into a queryable store, exposed by an API, and consumed by three presentation surfaces. Each layer is independently testable and replaceable — a deliberate choice for maintainability and a direct response to the technical debt that crippled MerMEId.

System Architecture — Music Works Catalogue Portal

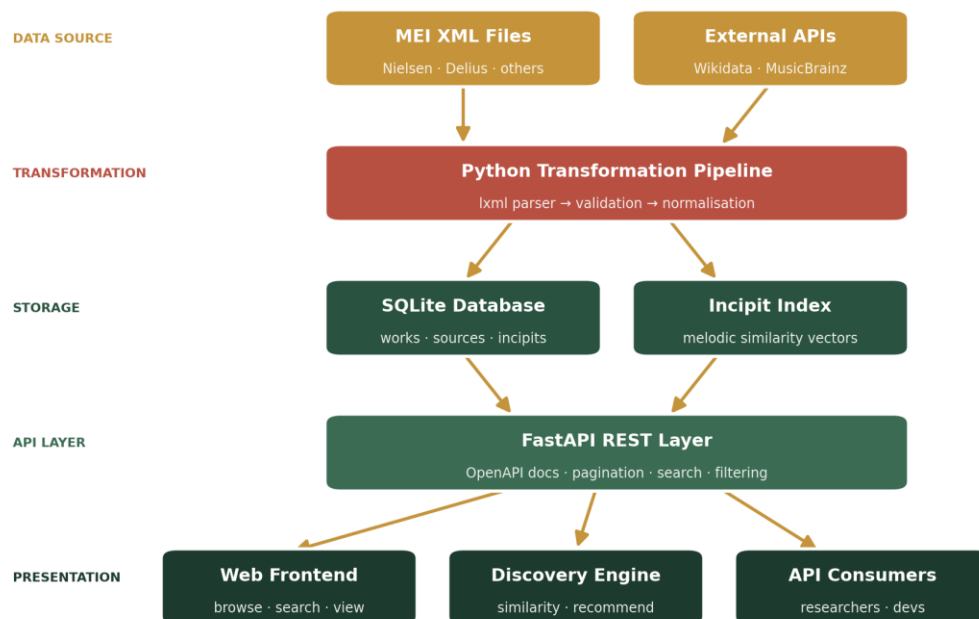


Figure 1: Layered system architecture with one-directional data flow and clear separation of concerns.

The transformation pipeline parses MEI XML with `lxml`, validates and normalises the data, and writes it to SQLite. The FastAPI layer exposes works, composers, sources, manuscripts, and incipits via documented JSON endpoints with search, filtering, and pagination. The presentation layer comprises a polished React frontend, the Works Discovery Engine, and direct API access. Code quality is enforced throughout by linting (`ruff`), static typing (`mypy`, `TypeScript`), and continuous integration.

6. Key Technologies and Methods

Layer	Technology	Justification
Transformation	Python + <code>lxml</code>	Robust XML parsing; dominant language for data pipelines
Storage	SQLite	Zero-config, portable, standard SQL; ideal for project scale
API	FastAPI + <code>Pydantic</code>	Auto-generated OpenAPI docs; type validation; async performance
Frontend	React + <code>TypeScript</code>	Component architecture, type safety, and a polished, maintainable UI

Layer	Technology	Justification
Discovery	music21 + numpy	music21 parses MEI incipits; numpy computes melodic similarity
Testing	pytest + Playwright	Unit/integration + end-to-end tests; addresses MerMEId's testing gap
Quality / CI	ruff, mypy, GitHub Actions	Linting, static typing, and continuous integration enforce professional coding style
Deployment	Docker	Reproducible; avoids the institutional dependency that broke MerMEId

Core methods: data modelling and transformation (mapping MEI's hierarchical XML to a relational schema), professional REST API design (OpenAPI conventions), information retrieval (full-text search), and music information retrieval (melodic similarity from incipits). Professional coding standards — type safety, linting, automated testing, and CI — are treated as first-class requirements, not optional extras.

7. Plan of Work

The project runs across the April–August semester in five overlapping phases. The table below sets out what each major task involves; the Gantt chart that follows shows their scheduling and dependencies.

Phase	Task	What it involves
Research (Apr)	Research & data analysis	Finalise literature review, gather MEI XML datasets, analyse the structure and completeness of the Nielsen and Delius catalogues
Design (Apr–May)	Schema, API & UI design	Design the relational schema mapping MEI to SQLite, define the OpenAPI contract, and produce wireframes and a design system for the interface
Build (May–Jul)	Pipeline, API, frontend	Implement the XML→SQLite transformation pipeline, the FastAPI REST layer, and the React frontend; integrate the second catalogue to prove catalogue-agnostic behaviour

Phase	Task	What it involves
Extend (Jul)	Discovery & linked data	Build the Works Discovery Engine (incipit similarity) and add Wikidata/MusicBrainz linked-data identifiers — scheduled as stretch goals
Evaluate (Jul–Aug)	Test, evaluate, write up	Run automated and usability testing, write the MEI evaluation report, and complete the final dissertation write-up

Four milestones mark key checkpoints: a working prototype (data + API) by the end of May, a feature-complete beta (full API and UI) in mid-June, feature completion (including discovery and linked data) in late July, and final submission in August.

Project Timeline — Music Works Catalogue Portal

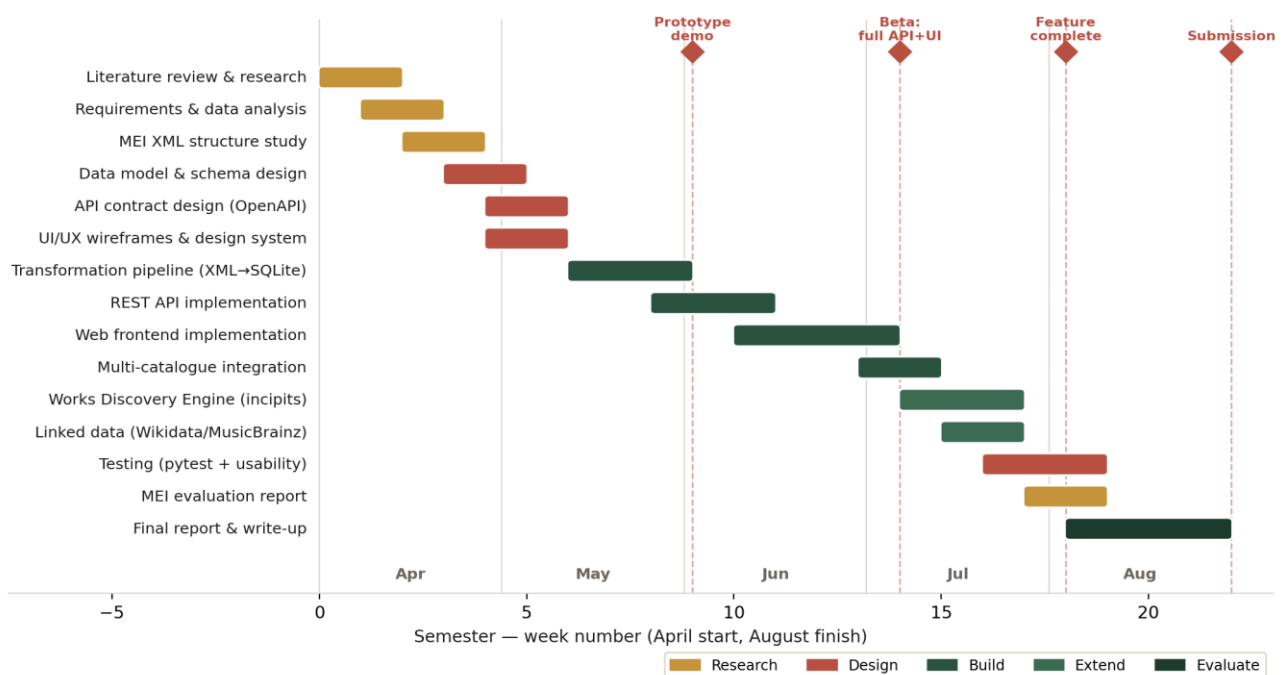


Figure 2: Gantt chart of the April–August semester showing tasks, phases, and milestones.

7.1 Feasibility and Risk Management

The plan is feasible because the core deliverables are deliberately front-loaded and decoupled from the ambitious extensions. The critical path runs through three tasks: the transformation pipeline, the REST API, and the web frontend. Together these constitute a complete, submittable project that

satisfies the pass and 2:1 criteria on their own — and they are scheduled to finish by mid-June, leaving roughly two months of buffer.

Risk is managed through a tiered scope. The Works Discovery Engine and linked-data integration are **stretch goals**, scheduled after the core system is complete. If any core task overruns, these extensions can be reduced or deferred without endangering a working submission. The main identified risks and mitigations are:

- MEI data more complex than expected — mitigated by the early XML structure study (weeks 2–4) and by starting with a reduced data model that can be enriched incrementally.
- Incipit similarity proves noisy or unworkable — mitigated by treating the Discovery Engine as a stretch goal; the core portal does not depend on it.
- Time pressure near submission — mitigated by the two-month buffer between the feature-complete beta and final submission, and by the front-loaded critical path.

Because each layer of the architecture is independent, partial completion still yields a working system: even if only the Nielsen catalogue and core API are delivered, the project meets its primary aims.

8. Testing and Evaluation

Evaluation combines technical correctness with user-centred assessment, since success depends on both a working and a usable system.

8.1 Automated testing

- Unit tests (pytest) for the transformation pipeline, verifying correct field extraction and graceful handling of edge cases — missing incipits, malformed records, multilingual metadata.
- API and end-to-end tests (pytest, Playwright) verifying responses, pagination, search accuracy, and correct error handling, all run automatically via continuous integration.

8.2 Data evaluation across datasets

The system is tested on three datasets: Carl Nielsen (~750 works, primary), Frederick Delius (~150 works, to verify catalogue-agnostic behaviour), and a synthetic edge-case set. Metrics: transformation completeness (percentage of source fields extracted), API accuracy against source XML, and cross-catalogue schema consistency.

8.3 Usability evaluation

As no user studies of thematic catalogue interfaces exist in the literature, a small usability evaluation will be run with representative users completing realistic tasks (e.g. 'find all works for solo piano composed before 1900'), measured by completion rate and qualitative feedback. Accessibility will be audited against WCAG 2.1 AA.

8.4 Defining success

The project succeeds if: (1) catalogue data is correctly transformed and served through a live, documented API; (2) a non-technical user can browse and search without XML knowledge; (3) at least two catalogues are explorable through one system; and (4) the MEI evaluation produces concrete, evidenced recommendations. These map directly to the brief's grading descriptors, with criterion (3) and the discovery engine targeting the first-class band.

References

- Bue, M., Gammert, J., Gubsch, C., Jettka, D. et al. (2023). 'Transitional MerMEId – Responding to Community Needs in Software and Sustainability.' TEI/MEC Conference, Paderborn.
- Krabbe, N. & Geertinger, A.T. (2012). 'MEI as a Basis for Thematic Catalogues.' RISM Conference on Music Documentation in Libraries, Scholarship, and Practice.
- Mathiak, B., Neuefeind, C., Schildkamp, P. et al. (2020). 'Sustainability Strategies for Digital Humanities Systems.' Digital Humanities Conference (DH2020), ADHO.
- Weigl, D.M., Goebel, W., Crawford, T. et al. (2019). 'Interweaving and Enriching Digital Music Collections for Scholarship, Performance, and Enjoyment.' DLFM, The Hague, pp. 84–88.

Declaration on the Use of AI

This document was written by the author. An AI assistant (Claude, Anthropic) was used to help polish wording, improve clarity, and refine the structure and formatting of the text. All design decisions, technical choices, research interpretation, and the overall direction of the project are the author's own. The cited sources were reviewed by the author.